

CodeBounty PRD

1. Product Summary

CodeBounty is a Next.js coding practice platform that helps users learn data structures and algorithms by recognizing problem-solving patterns, practicing inside curated learning paths, receiving AI hints, and joining host-created coding sessions.

The tutorial project provides the execution-heavy base: authentication, problem creation, Judge0 code execution, submissions, playlists, and profile stats. CodeBounty builds on top of that foundation by shifting the product from a generic LeetCode clone into a structured practice platform for pattern-based DSA learning, peer challenges, classroom-style quizzes, and timed contests.

2. Problem Statement

Most learners do not struggle only because they lack problems. They struggle because they do not know which pattern a problem belongs to, when to apply that pattern, or how their practice connects across topics. Generic problem tables encourage random solving. CodeBounty should guide users toward pattern mastery.

Groups also need a lightweight way to run timed practice without requiring a formal classroom structure. A college teacher, coding club lead, friend group, or solo learner should all be able to create a session, share an ID and password, and review results.

3. Goals

- Help learners practice problems by patterns such as Sliding Window, Two Pointers, Binary Search, Trees, Graphs, DP, Greedy, and Backtracking.
- Preserve the tutorial's strong base: auth, admin problem creation, Judge0 execution, submissions, solved tracking, playlists, and profile stats.
- Add AI hints that teach direction without giving away the full answer immediately.
- Add user-hosted sessions for quizzes, contests, group practice, and mock tests.
- Show progress through pattern-wise stats and heatmaps.
- Make the final project resume-worthy by having a clear product identity, real workflows, and measurable learning outcomes.

4. Non-Goals

- Rebuilding Judge0, Clerk, or Prisma flows from scratch after the tutorial.
- Creating a full commercial LMS or classroom management system.
- Supporting real-money rewards despite the “CodeBounty” name.
- Building advanced proctoring, plagiarism detection, or live classroom video features in v1.
- Creating a mobile-native app.

5. Product Principles

- Keep identity simple: every signed-in person is a user first.
- Do not lock collaboration behind a teacher role.
- Model timed practice as a reusable session primitive.
- Let context define usage: the same session can be a classroom quiz, friend contest, coding club round, or solo timed drill.
- Keep platform power separate from product participation: admins moderate and manage official content, while users practice, host, join, and compete.
- Build features around ownership and permissions, not titles. A session owner can manage that session because they created it, not because they are labeled as a teacher.
- Make the product meaningful for learning first, then social and competitive second.

6. Access Model

CodeBounty should use a simple global role model:

- **USER** : can practice, create playlists, request hints, create sessions, join sessions, and view permitted leaderboards.
- **ADMIN** : can do everything a user can do, plus manage official problems, patterns, moderation, and platform settings.

Session-level permissions are separate from global roles:

- **HOST** : the user who created the session.
- **PARTICIPANT** : a user who joined with a valid join code and password.

This keeps the system scalable. A teacher does not need a special account to run a quiz, and a friend group does not need fake teacher permissions to practice together.

7. Target Users

Learner

A user preparing for internships, placements, coding interviews, college exams, or personal improvement. They want a guided path instead of a random list of problems.

Session Host

Any signed-in user who wants to create a timed practice session, share it with others, and review scores. This can be a teacher, mentor, coding club lead, friend, senior, or study group member.

Platform Admin

The owner of the app. They manage users, problem quality, platform settings, and seeded content.

8. Core User Stories

User Stories

- As a user, I can browse problems by pattern so I know what skill I am practicing.
- As a user, I can filter problems by difficulty, pattern, tags, solved status, and search text.
- As a user, I can open a problem, read examples and constraints, write code, run test cases, and submit.
- As a user, I can request an AI hint when I am stuck during normal practice.
- As a user, I can see my solved count, submission history, playlists, and pattern-wise progress.
- As a user, I can create a timed session from existing problems and share its join details.
- As a user, I can join a session using a session ID and password.
- As a user, I can view the leaderboard for sessions I host or join, depending on the session settings.

Host Stories

- As a host, I can create a session from existing problems.
- As a host, I can set title, description, start time, end time, duration, visibility, password, and hint policy.
- As a host, I can share a session ID and password with participants.
- As a host, I can view participants, submissions, scores, and leaderboard.
- As a host, I can close, archive, or duplicate a session.

Admin Stories

- As an admin, I can manage platform users and moderation.
- As an admin, I can manage patterns, tags, problems, sessions, and platform content.

- As an admin, I can seed sample problems for demo and resume review.

9. Tutorial Foundation To Keep

The VTT analysis shows the tutorial already covers these base modules:

- Next.js project setup with Shadcn UI.
- Prisma and PostgreSQL setup, including Dockerized database flow.
- Clerk authentication with GitHub OAuth and custom auth pages.
- User onboarding into the database through Prisma and server actions.
- Responsive navbar, homepage, dark/light mode, and role-based UI.
- Judge0 setup through local self-hosting or RapidAPI.
- Admin problem creation with Prisma schema.
- Judge0 validation before saving test cases.
- Problem table with filters and search.
- Problem solving page with left/right layout and code execution.
- Submission and test case schemas.
- User-specific submission history.
- Playlist creation and adding problems to playlists.
- Profile page with user info, stats, submissions, and playlists.

This should be implemented as-is first. Customization starts after the tutorial foundation is stable.

10. Product Differentiators

Pattern-First Learning

Problems are organized around reusable problem-solving patterns instead of only broad topics. A problem may belong to one primary pattern and multiple secondary tags.

Examples:

- Sliding Window
- Two Pointers
- Prefix Sum
- Binary Search on Answer
- Fast and Slow Pointers
- Monotonic Stack
- Heap / Priority Queue
- BFS / DFS
- Dynamic Programming
- Greedy

- Backtracking

AI Hint Ladder

The AI hint feature should not immediately reveal the solution. It should provide progressive hints:

- Hint 1: Identify the pattern.
- Hint 2: Explain the key observation.
- Hint 3: Suggest algorithm steps.
- Hint 4: Provide pseudocode.
- Hint 5: Explain complexity and common mistakes.

Host-Created Sessions

Any user can create a timed set of problems for practice, homework, friend contests, coding club rounds, or mock interviews. This gives the product a meaningful collaboration angle without forcing users into fixed identities.

Pattern Progress Heatmap

Instead of showing only total solved problems, the profile shows progress by pattern. Users can see strengths and weak areas.

11. Functional Requirements

11.1 Authentication And Permissions

- Use Clerk for authentication.
- Persist users in Prisma after first login.
- Support roles:
 - USER
 - ADMIN
- Protect admin routes.
- Let every signed-in user create and join sessions.
- Enforce session permissions through ownership, membership, password, timing, and visibility.
- Show owner-only actions for sessions created by the current user.

11.2 Problem Management

- Admins can create and manage the official problem bank.
- In v1, normal users create sessions from existing problems instead of creating public problems.
- In v2, trusted users may be allowed to submit community problems for admin review.
- Problems include:

- title
- slug
- description
- difficulty
- constraints
- examples
- starter code per language
- reference solution
- test cases
- primary pattern
- tags
- visibility
- creator
- Judge0 validates official solutions against test cases before saving.
- Problems can be edited, archived, and searched.

11.3 Pattern System

- Admins can create and edit patterns.
- Each pattern includes:
 - name
 - slug
 - description
 - order
 - icon/color metadata
 - recommended prerequisites
- Problems link to one primary pattern and optional tags.
- Pattern pages show:
 - explanation
 - problem list
 - solved count
 - recommended order

11.4 Problem Solving

- Users can write code in a browser editor.
- Users can run code against sample test cases.
- Users can submit code against hidden test cases.
- Submission result stores:
 - status
 - runtime

- memory
- source code
- language
- per-testcase result
- error output where applicable
- Successful submissions update solved tracking.

11.5 AI Hints

- Users can request hints on a problem during normal practice.
- The system passes safe context to Gemini:
 - problem title
 - description
 - constraints
 - examples
 - pattern if allowed
 - user's current code optionally
- AI returns progressive hints.
- Hints are stored so repeated clicks do not unnecessarily call the API.
- The app should rate-limit hint requests per user.
- For session mode, the session host can disable AI hints.

11.6 Playlists / Sheets

- Users can create personal playlists.
- Users can add and remove problems from playlists.
- Playlists can become "sheets" for pattern practice.
- In a later version, users can publish playlists or share them with a group.

11.7 Sessions / Quizzes / Contests

- Any signed-in user can create a session from existing problems.
- Session fields:
 - title
 - description
 - start time
 - end time
 - duration
 - problems
 - visibility
 - join password
 - hint policy

- Users join sessions with session ID and password.
- A session can be `PRACTICE` , `QUIZ` , `CONTEST` , or `MOCK_INTERVIEW` .
- The platform records submissions inside a session attempt context.
- Session results show:
 - participant
 - solved count
 - score
 - last submission time
 - per-problem status
- Hosts can view all participant results.
- Participants can view leaderboard if the session visibility allows it.

11.8 Profile And Analytics

- Profile shows:
 - total solved
 - solved by difficulty
 - solved by pattern
 - recent submissions
 - playlists
 - session attempts
- Pattern heatmap shows weak and strong areas.
- A user can open a pattern and continue from unsolved recommended problems.

12. Recommended Data Model Additions

The tutorial schema should be extended after completion.

User

- `id`
- `clerkUserId`
- `email`
- `name`
- `imageUrl`
- `role`
- timestamps

Problem

Existing tutorial problem model should be extended with:

- slug
- primaryPatternId
- createdById
- visibility
- isArchived

Pattern

- id
- name
- slug
- description
- order
- color
- icon
- timestamps

Tag

- id
- name
- slug

ProblemTag

- problemId
- tagId

Submission

Existing tutorial submission model should be extended with:

- sessionAttemptId
- isAccepted
- score

SolvedProblem

- userId
- problemId
- solvedAt
- bestSubmissionId

AIHint

- `id`
- `userId`
- `problemId`
- `level`
- `promptHash`
- `content`
- `timestamps`

Session

- `id`
- `title`
- `slug`
- `description`
- `hostId`
- `joinCode`
- `passwordHash`
- `type`
- `startTime`
- `endTime`
- `durationMinutes`
- `visibility`
- `allowHints`
- `leaderboardVisibility`
- `status`
- `timestamps`

SessionProblem

- `sessionId`
- `problemId`
- `order`
- `points`

SessionAttempt

- `id`
- `sessionId`
- `userId`
- `startedAt`

- `submittedAt`
- `score`
- `status`

SessionParticipant

- `sessionId`
- `userId`
- `joinedAt`
- `displayName`
- `status`

13. Key Workflows

Practice Workflow

1. User signs in.
2. User opens Patterns.
3. User selects Sliding Window.
4. User sees explanation, progress, and ordered problems.
5. User opens a problem.
6. User writes code and runs sample tests.
7. User requests an AI hint if stuck.
8. User submits.
9. Accepted submission updates solved status and pattern progress.

Admin Problem Creation Workflow

1. Admin opens Create Problem.
2. Admin enters problem details, starter code, examples, and test cases.
3. Admin selects primary pattern and tags.
4. Admin submits.
5. Judge0 validates reference solution.
6. Problem is saved only if validation passes.

Session Hosting Workflow

1. User opens Sessions.
2. User creates a session.
3. User selects problems from the existing problem bank.
4. User configures type, timing, password, scoring, and hint policy.

5. User shares the session ID and password.
6. Participants join the session.
7. Submissions are tracked under session attempts.
8. Host reviews leaderboard and per-participant performance.

14. UX Requirements

- The app should feel like a focused learning dashboard, not a generic marketing page.
- First useful screen after login should show practice progress and recommended patterns.
- Problem list should be dense, searchable, and filterable.
- Pattern pages should be visually clear, with progress indicators and ordered problem rows.
- Problem solving page should prioritize editor usability, output clarity, and fast feedback.
- AI hint UI should be a small contextual panel or drawer, not a disruptive modal.
- Session dashboard should be utilitarian: hosted sessions, joined sessions, participants, results, and actions.

15. Suggested Routes

- `/`
- `/dashboard`
- `/patterns`
- `/patterns/[slug]`
- `/problems`
- `/problems/[id]` or `/problems/[slug]`
- `/playlists`
- `/profile`
- `/admin/problems/new`
- `/admin/problems/[id]/edit`
- `/sessions`
- `/sessions/new`
- `/sessions/join`
- `/sessions/[joinCode]`
- `/sessions/[joinCode]/attempt`
- `/sessions/[joinCode]/leaderboard`

16. Success Metrics

- A user can solve and submit at least one problem end-to-end.
- Solved status appears correctly in the problem table, profile, and pattern page.
- A user can request progressive AI hints without receiving the full answer too early.
- A user can create a session from existing problems.

- Another user can join that session with ID and password.
- Session submissions are associated with the correct session attempt.
- Host can view leaderboard and participant results.
- Profile shows accurate pattern-wise progress.
- The README can clearly explain why CodeBounty is more than a LeetCode clone.

17. MVP Scope

MVP should include:

- Tutorial base fully completed.
- Pattern model and pattern-based problem filtering.
- Pattern pages with solved progress.
- Gemini AI hint ladder.
- User-created sessions with ID/password join flow.
- Session attempt, scoring, and leaderboard tracking.
- Profile pattern progress.
- Strong README and seeded demo problems.

18. V2 Ideas

- Public pattern sheets.
- Classrooms and teacher-student enrollment.
- Badges and streaks.
- Editorial solution pages.
- Discussion threads per problem.
- AI code review after submission.
- Advanced contest leaderboard with tie-breaking rules.
- Public session discovery.
- Team sessions.
- Import problems from CSV.